《计算机图形学及人机交互》

作 业 报 告

学　　号	2023112621
姓　　名	程泽凯
年　　级	2023 级
专　　业	计算机科学与技术
院　　系	计算机与人工智能学院

2025~2026 第 1 学期

目录

1 三维图形顶点数据结构设计 1

1.1 题目要求 . . . 1
1.2 设计思路 . . . 1
1.3 数据结构定义 . . . 1
1.3.1 基础向量结构 . . . 1
1.3.2 顶点数据结构 . . . 2
1.3.3 辅助数据结构 . . . 2
1.4 各字段详细说明 . . . 4
1.5 光照计算原理 . . . 4
1.5.1 Phong 光照模型 . . . 4
1.5.2 阴影映射算法 . . . 5
1.5.3 法线贴图与粗糙度 . . . 5

2 三阶 B 样条曲面设计与实现 6

2.1 题目要求 . . . 6
2.2 B 样条曲面理论基础 . . . 6
2.2.1 B 样条基函数 . . . 6
2.2.2 B 样条曲面定义 . . . 6
2.2.3 节点向量 . . . 6
2.3 三角形网格生成 . . . 7
2.3.1 曲面离散化 . . . 7
2.3.2 三角形索引生成 . . . 7
2.4 Rust 代码实现 . . . 7
2.4.1 数据结构定义 . . . 7
2.4.2 B 样条曲面结构 . . . 9
2.4.3 基函数计算 . . . 10
2.4.4 网格生成 . . . 12
2.5 程序运行结果 . . . 13

3 正方体纹理坐标设计 16

3.1 题目要求 . . . 16
3.2 问题分析 . . . 16
3.2.1 纹理图片布局 . . . 16
3.2.2 正方体顶点定义 . . . 16
3.3 各面纹理坐标设计 . . . 18
3.3.1 纹理坐标映射原则 . . . 18

3.3.2 六个面的详细设计 18
3.4 纹理坐标汇总表 20
3.5 代码实现示例 20
3.6 注意事项 24

1 三维图形顶点数据结构设计

1.1 题目要求

请根据下面的要求分别设计相应的数据结构用于存储某三维图形的顶点：

1. 该三维图形的颜色由顶点的颜色确定。
2. 在设置不同光照的情况下，该图形的颜色也会发生改变。
3. 在设置不同纹理的情况下，该图形外表面的粗糙程度也会发生改变。
4. 在不同光照情况下，可显示出该图形的阴影。

1.2 设计思路

为了满足上述四个要求，需要在顶点数据结构中包含以下关键信息：

- **位置坐标**：顶点在三维空间中的位置，是最基本的属性。
- **顶点颜色**：满足要求 (1)，通过顶点颜色插值实现图形着色。
- **法向量**：满足要求 (2) 和 (4)，用于光照计算（漫反射、镜面反射）和阴影判断。
- **纹理坐标**：满足要求 (3)，用于采样纹理贴图。
- **切线空间向量**：满足要求 (3)，用于法线贴图计算，实现表面粗糙度效果。

1.3 数据结构定义

1.3.1 基础向量结构

```rust
/// 三维向量结构，用于位置、颜色、法向量等
#[derive(Debug, Clone, Copy)]
pub struct Vec3 {
    pub x: f32,    // X分量
    pub y: f32,    // Y分量
    pub z: f32,    // Z分量
}

/// 二维向量结构，用于纹理坐标
#[derive(Debug, Clone, Copy)]
pub struct Vec2 {
    pub u: f32,    // 纹理U坐标（水平方向）
```

```
    pub v: f32,     // 纹理V坐标 (垂直方向)
}
```

Listing 1: 基础向量结构定义

1.3.2 顶点数据结构

```
/// 顶点数据结构
#[derive(Debug, Clone, Copy)]
pub struct Vertex {
    // 基础属性
    pub position: Vec3,        // 顶点位置坐标 (x, y, z)

    // 要求(1): 顶点颜色
    pub color: Vec3,           // 顶点颜色 (R, G, B), 范围[0,1]

    // 要求(2)(4): 光照和阴影计算
    pub normal: Vec3,          // 顶点法向量, 用于光照计算

    // 要求(3): 纹理映射
    pub tex_coord: Vec2,       // 纹理坐标 (u, v)

    // 要求(3): 切线空间, 用于法线贴图实现粗糙度
    pub tangent: Vec3,         // 切线向量
    pub bitangent: Vec3,       // 副切线向量
}
```

Listing 2: 完整顶点数据结构

1.3.3 辅助数据结构

为了完整实现光照和材质效果，还需要定义以下辅助结构：

```
/// 材质属性结构
#[derive(Debug, Clone, Copy)]
pub struct Material {
    pub ambient: Vec3,         // 环境光反射系数 (Ka)
    pub diffuse: Vec3,         // 漫反射系数 (Kd)
    pub specular: Vec3,        // 镜面反射系数 (Ks)
    pub shininess: f32,        // 高光指数 (n)
    pub roughness: f32,        // 粗糙度, 范围[0,1]
```

```rust
    pub metallic: f32,        // 金属度，范围[0,1]
}

/// 光源类型枚举
#[derive(Debug, Clone, Copy)]
pub enum LightType {
    Point,        // 点光源
    Directional, // 平行光
    Spot,         // 聚光灯
}

/// 光源结构
#[derive(Debug, Clone, Copy)]
pub struct Light {
    pub position: Vec3,       // 光源位置
    pub direction: Vec3,      // 光源方向（用于平行光和聚光灯）
    pub color: Vec3,          // 光源颜色
    pub intensity: f32,       // 光照强度
    pub light_type: LightType, // 光源类型
}

/// 阴影贴图信息
pub struct ShadowMap {
    pub depth_map: Vec<f32>,      // 深度贴图数据
    pub width: usize,             // 贴图宽度
    pub height: usize,            // 贴图高度
    pub light_matrix: [f32; 16], // 光源空间变换矩阵
}
```

Listing 3: 材质和光源结构

1.4 各字段详细说明

表 1: 顶点数据结构字段说明

字段	类型	对应要求	功能说明
position	Vec3	基础	存储顶点在世界坐标系中的位置 (x, y, z)
color	Vec3	要求 (1)	存储顶点 RGB 颜色，通过重心坐标插值确定片元颜色
normal	Vec3	要求 (2)(4)	存储顶点法向量，用于计算光照强度和阴影
texCoord	Vec2	要求 (3)	存储纹理坐标 (u, v)，用于采样纹理贴图
tangent	Vec3	要求 (3)	切线向量，与 normal、bitangent 构成 TBN 矩阵
bitangent	Vec3	要求 (3)	副切线向量，用于法线贴图的切线空间变换

1.5 光照计算原理

1.5.1 Phong 光照模型

满足要求 (2)，使用 Phong 光照模型计算顶点颜色：

$$I = I_a \cdot K_a + I_d \cdot K_d \cdot (\vec{N} \cdot \vec{L}) + I_s \cdot K_s \cdot (\vec{R} \cdot \vec{V})^n \tag{1}$$

其中：

- I_a, I_d, I_s：环境光、漫反射光、镜面反射光强度
- K_a, K_d, K_s：材质的环境光、漫反射、镜面反射系数
- $\vec{N}$：顶点法向量（来自 normal 字段）
- $\vec{L}$：光源方向向量
- $\vec{R}$：反射光方向向量
- $\vec{V}$：视线方向向量
- n：高光指数

1.5.2 阴影映射算法

满足要求 (4)，使用 Shadow Mapping 算法实现阴影：

1. 从光源视角渲染场景，生成深度贴图（Shadow Map）
2. 渲染场景时，将每个片元变换到光源空间
3. 比较片元深度与深度贴图中的值，判断是否在阴影中

1.5.3 法线贴图与粗糙度

满足要求 (3)，通过 TBN 矩阵实现法线贴图：

$$TBN = \begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix} \tag{2}$$

其中 $\vec{T}$（tangent）、$\vec{B}$（bitangent）、$\vec{N}$（normal）构成切线空间的正交基。通过法线贴图可以在不增加几何复杂度的情况下模拟表面粗糙度变化。

2 三阶 B 样条曲面设计与实现

2.1 题目要求

编写程序，设计一个三阶的 B 样条曲面，在给定控制点的情况下，计算出 B 样条曲面上的点（设 u，v 方向的曲面精度可通过参数设置），然后将这些点保存到数组中。如果该 B 样条曲面以三角面为基本图元进行存储，计算出三角形顶点的索引，并保存到数组中。采用 Rust 语言。

2.2 B 样条曲面理论基础

2.2.1 B 样条基函数

B 样条基函数采用 Cox-de Boor 递推公式定义。设节点向量为 $\{t_0, t_1, \ldots, t_m\}$，则 p 次 B 样条基函数定义为：

零次基函数（$p = 0$）：

$$N_{i,0}(t) = \begin{cases} 1 & \text{若} t_i \le t < t_{i+1} \\ 0 & \text{其他} \end{cases} \tag{3}$$

递推公式（$p \ge 1$）：

$$N_{i,p}(t) = \frac{t - t_i}{t_{i+p} - t_i} N_{i,p-1}(t) + \frac{t_{i+p+1} - t}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(t) \tag{4}$$

其中，当分母为零时，约定该项为零。

2.2.2 B 样条曲面定义

给定 $m \times n$ 个控制点 $\mathbf{P}_{i,j}$（$i = 0, 1, \ldots, m-1$；$j = 0, 1, \ldots, n-1$），p 次 $\times$ q 次 B 样条曲面定义为：

$$\mathbf{S}(u, v) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} N_{i,p}(u) \cdot N_{j,q}(v) \cdot \mathbf{P}_{i,j} \tag{5}$$

对于三阶 B 样条曲面，$p = q = 3$（即 3 次 B 样条）。

2.2.3 节点向量

对于 n 个控制点和 p 次 B 样条，节点向量包含 $n + p + 1$ 个节点。本实现采用均匀开放节点向量：

$$t_i = \begin{cases} 0 & i \le p \\ \frac{i-p}{n-p} & p < i < n \\ 1 & i \ge n \end{cases} \tag{6}$$

2.3 三角形网格生成

2.3.1 曲面离散化

将参数域 $[0,1] \times [0,1]$ 按照给定精度离散化：

- u 方向细分为 u_{res} 段，共 $u_{res}+1$ 个采样点
- v 方向细分为 v_{res} 段，共 $v_{res}+1$ 个采样点
- 总顶点数：$(u_{res}+1) \times (v_{res}+1)$

2.3.2 三角形索引生成

每个网格单元划分为两个三角形。对于网格点 (i,j)，其在顶点数组中的索引为：

$$idx(i,j) = i \times (v_{res}+1) + j \tag{7}$$

两个三角形的顶点索引为：

- **三角形 1**：$(idx(i,j),\ idx(i,j+1),\ idx(i+1,j+1))$
- **三角形 2**：$(idx(i,j),\ idx(i+1,j+1),\ idx(i+1,j))$

总三角形数量：$2 \times u_{res} \times v_{res}$

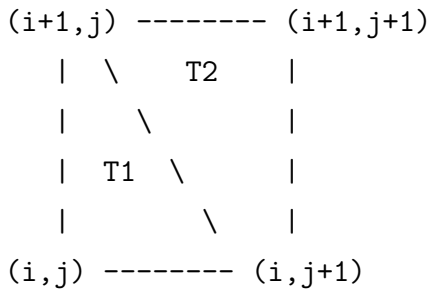


图 1: 网格单元三角形划分示意图

2.4 Rust 代码实现

2.4.1 数据结构定义

```rust
/// 三维点结构
#[derive(Debug, Clone, Copy)]
pub struct Point3 {
    pub x: f64,
    pub y: f64,
```

```rust
    pub z: f64,
}

impl Point3 {
    /// 创建新的三维点
    pub fn new(x: f64, y: f64, z: f64) -> Self {
        Self { x, y, z }
    }

    /// 创建零点
    pub fn zero() -> Self {
        Self { x: 0.0, y: 0.0, z: 0.0 }
    }
}

/// 实现加法运算
impl std::ops::Add for Point3 {
    type Output = Self;
    fn add(self, other: Self) -> Self {
        Self {
            x: self.x + other.x,
            y: self.y + other.y,
            z: self.z + other.z,
        }
    }
}

/// 实现标量乘法
impl std::ops::Mul<f64> for Point3 {
    type Output = Self;
    fn mul(self, scalar: f64) -> Self {
        Self {
            x: self.x * scalar,
            y: self.y * scalar,
            z: self.z * scalar,
        }
    }
}
```

Listing 4: 三维点结构

2.4.2 B 样条曲面结构

```rust
/// B样条曲面结构
pub struct BSplineSurface {
    control_points: Vec<Vec<Point3>>,  // m x n 控制点网格
    knots_u: Vec<f64>,                  // u方向节点向量
    knots_v: Vec<f64>,                  // v方向节点向量
    degree: usize,                      // 曲线次数
}

impl BSplineSurface {
    /// 创建三阶B样条曲面
    /// 参数: control_points - m x n 控制点网格
    pub fn new(control_points: Vec<Vec<Point3>>) -> Self {
        let m = control_points.len();
        let n = control_points[0].len();
        let degree = 3; // 三阶B样条, 次数为3

        // 生成均匀开放节点向量
        let knots_u = Self::generate_uniform_knots(m, degree);
        let knots_v = Self::generate_uniform_knots(n, degree);

        Self {
            control_points,
            knots_u,
            knots_v,
            degree,
        }
    }

    /// 生成均匀开放节点向量
    fn generate_uniform_knots(n: usize, degree: usize) -> Vec<f64
        > {
        let num_knots = n + degree + 1;
        let mut knots = Vec::with_capacity(num_knots);

        for i in 0..num_knots {
            if i <= degree {
                knots.push(0.0);  // 前degree+1个节点为0
            } else if i >= num_knots - degree - 1 {
```

```
                knots.push(1.0);  // 后degree+1个节点为1
            } else {
                // 中间节点均匀分布
                let val = (i - degree) as f64
                        / (num_knots - 2 * degree - 1) as f64;
                knots.push(val);
            }
        }
        knots
    }
}
```

Listing 5: B 样条曲面结构

2.4.3 基函数计算

```
impl BSplineSurface {
    /// Cox-de Boor递推公式计算B样条基函数
    /// 参数: i - 基函数索引, p - 次数, t - 参数值, knots - 节点
       向量
    fn basis_function(&self, i: usize, p: usize, t: f64,
                      knots: &[f64]) -> f64 {
        // 零次基函数
        if p == 0 {
            if knots[i] <= t && t < knots[i + 1] {
                return 1.0;
            }
            // 处理 t = 1.0 的边界情况
            if (t - 1.0).abs() < 1e-10
               && (knots[i + 1] - 1.0).abs() < 1e-10 {
                return 1.0;
            }
            return 0.0;
        }

        // 递推计算
        let mut result = 0.0;

        // 第一项
        let denom1 = knots[i + p] - knots[i];
```

```
24          if denom1.abs() > 1e-10 {
25              result += (t - knots[i]) / denom1
26                      * self.basis_function(i, p - 1, t, knots);
27          }
28
29          // 第二项
30          let denom2 = knots[i + p + 1] - knots[i + 1];
31          if denom2.abs() > 1e-10 {
32              result += (knots[i + p + 1] - t) / denom2
33                      * self.basis_function(i + 1, p - 1, t, knots)
                        ;
34          }
35
36          result
37      }
38
39      /// 计算曲面上一点 S(u,v)
40      pub fn evaluate(&self, u: f64, v: f64) -> Point3 {
41          let m = self.control_points.len();
42          let n = self.control_points[0].len();
43          let mut point = Point3::zero();
44
45          // 双重求和
46          for i in 0..m {
47              let basis_u = self.basis_function(i, self.degree,
48                                                u, &self.knots_u);
49              for j in 0..n {
50                  let basis_v = self.basis_function(j, self.degree,
51                                                    v, &self.
                                                       knots_v);
52                  let weight = basis_u * basis_v;
53                  point = point + self.control_points[i][j] *
                        weight;
54              }
55          }
56
57          point
58      }
59  }
```

Listing 6: Cox-de Boor 递推算法

2.4.4 网格生成

```rust
impl BSplineSurface {
    /// 生成曲面点和三角形索引
    /// 参数: u_resolution - u方向细分数
    ///       v_resolution - v方向细分数
    /// 返回: (顶点数组, 三角形索引数组)
    pub fn generate_mesh(&self, u_resolution: usize,
                         v_resolution: usize)
        -> (Vec<Point3>, Vec<[usize; 3]>)
    {
        let mut vertices = Vec::new();
        let mut indices = Vec::new();

        // 生成曲面上的点
        for i in 0..=u_resolution {
            let u = i as f64 / u_resolution as f64;
            for j in 0..=v_resolution {
                let v = j as f64 / v_resolution as f64;
                vertices.push(self.evaluate(u, v));
            }
        }

        // 生成三角形索引
        let cols = v_resolution + 1;
        for i in 0..u_resolution {
            for j in 0..v_resolution {
                let idx = i * cols + j;

                // 三角形1: 左下-右下-右上
                indices.push([idx, idx + 1, idx + cols + 1]);

                // 三角形2: 左下-右上-左上
                indices.push([idx, idx + cols + 1, idx + cols]);
            }
        }
```

```
35
36              (vertices, indices)
37          }
38      }
```

Listing 7: 曲面网格生成

2.5 程序运行结果

使用 4×4 控制点网格，u 和 v 方向各细分 10 段，程序运行结果如图2和图3所示。

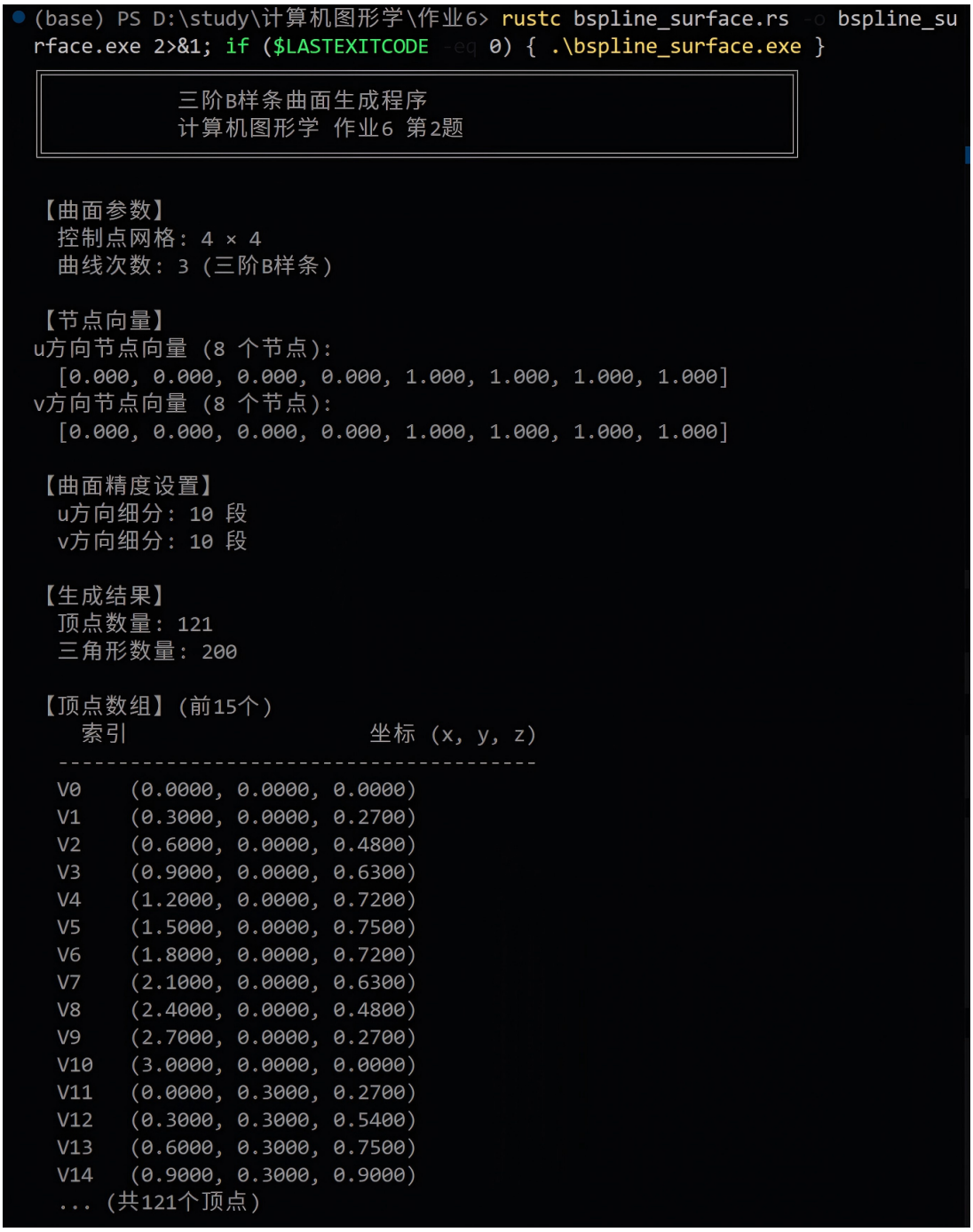


图 2: 三阶 B 样条曲面程序运行结果（一）

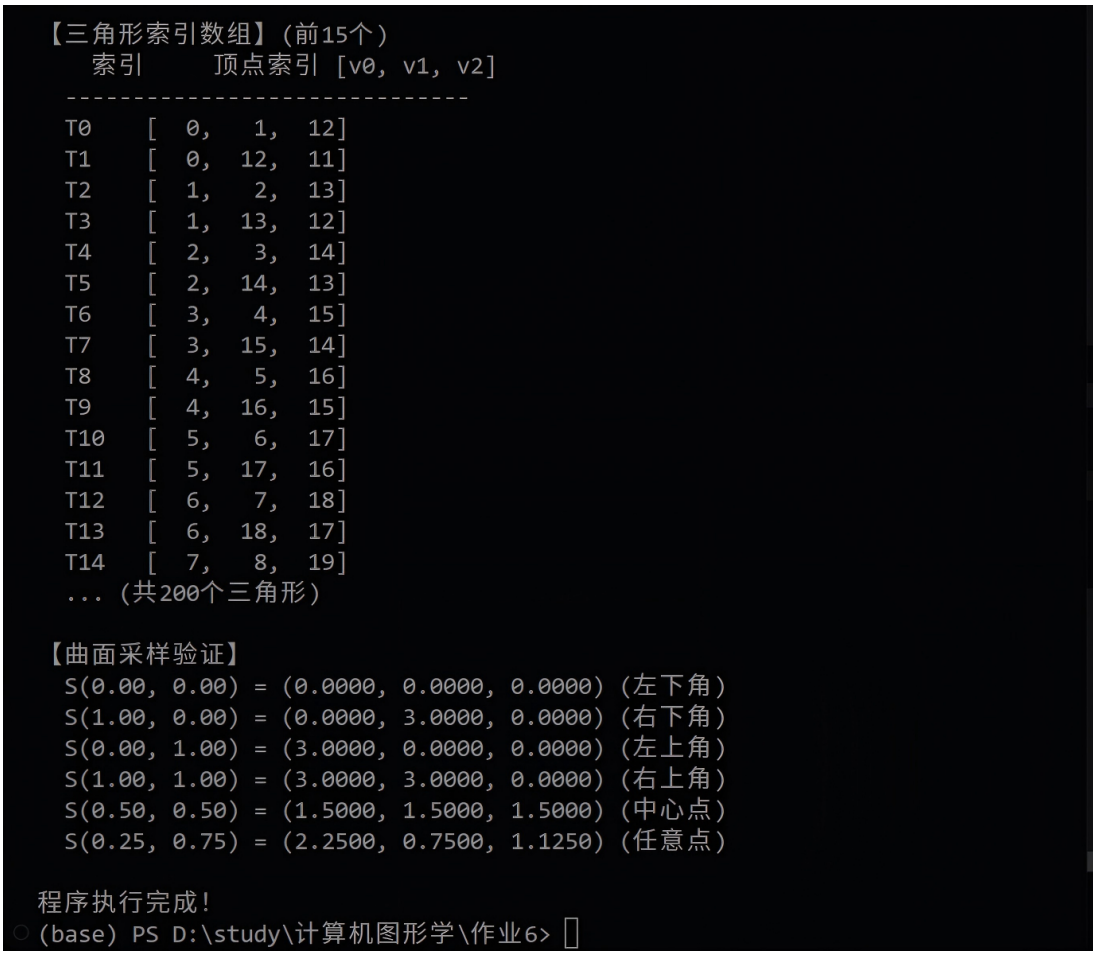


图 3: 三阶 B 样条曲面程序运行结果（二）

程序输出的主要内容包括：

- 曲面参数信息：控制点网格大小、曲线次数
- 节点向量：u 方向和 v 方向的均匀开放节点向量
- 生成结果统计：顶点数量 121 个，三角形数量 200 个
- 顶点数组：存储曲面上离散点的三维坐标
- 三角形索引数组：存储三角形顶点的索引关系
- 曲面采样验证：验证曲面角点和中心点的坐标值

3 正方体纹理坐标设计

3.1 题目要求

一张图片上有 8 个不同的数字，每个数字的宽度和长度均相同。现希望将其前 6 个数字作为纹理贴到一个正方体的六个面上，使每个面上分别显示出不同的数字，请问如何设计该正方形每个顶点的纹理坐标。

3.2 问题分析

3.2.1 纹理图片布局

假设 8 个数字在图片中水平排列（8×1 布局），则：

- 图片总宽度归一化为 1.0
- 每个数字占据宽度：$\frac{1}{8} = 0.125$
- 图片高度归一化为 1.0
- 第 n 个数字（$n = 0, 1, \ldots, 7$）的纹理坐标范围：

$$u \in \left[\frac{n}{8}, \frac{n+1}{8}\right], \quad v \in [0, 1] \tag{8}$$

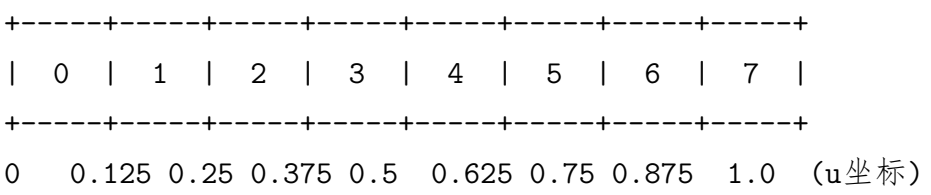


图 4: 纹理图片布局示意图（8 个数字水平排列）

3.2.2 正方体顶点定义

设正方体边长为 2，中心位于原点，则 8 个顶点坐标为：

表 2: 正方体顶点坐标

顶点	X	Y	Z
V_0	-1	-1	-1
V_1	$+1$	-1	-1
V_2	$+1$	$+1$	-1
V_3	-1	$+1$	-1
V_4	-1	-1	$+1$
V_5	$+1$	-1	$+1$
V_6	$+1$	$+1$	$+1$
V_7	-1	$+1$	$+1$

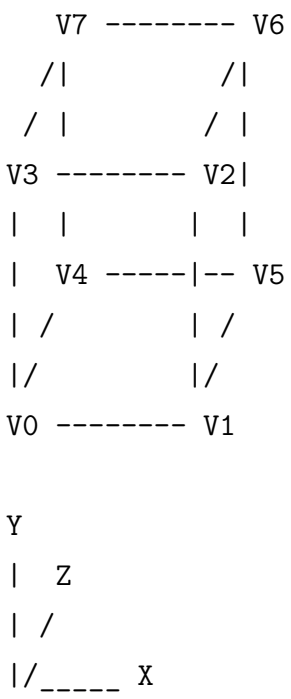


图 5: 正方体顶点编号示意图

3.3 各面纹理坐标设计

3.3.1 纹理坐标映射原则

对于每个面，需要将四个顶点映射到对应数字的纹理区域。设第 n 个数字的纹理坐标边界为：

$$u_{min} = \frac{n}{8} \tag{9}$$

$$u_{max} = \frac{n+1}{8} \tag{10}$$

$$v_{min} = 0 \tag{11}$$

$$v_{max} = 1 \tag{12}$$

四个角的纹理坐标为：

- 左下角：(u_{min}, v_{min})
- 右下角：(u_{max}, v_{min})
- 右上角：(u_{max}, v_{max})
- 左上角：(u_{min}, v_{max})

3.3.2 六个面的详细设计

面 1：前面（+Z 方向）- 数字 0

- 顶点顺序：$V_4 \to V_5 \to V_6 \to V_7$（逆时针）
- 数字编号：0
- 纹理范围：$u \in [0, 0.125]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_4	$(-1, -1, +1)$	$(0.000, 0)$	左下
V_5	$(+1, -1, +1)$	$(0.125, 0)$	右下
V_6	$(+1, +1, +1)$	$(0.125, 1)$	右上
V_7	$(-1, +1, +1)$	$(0.000, 1)$	左上

面 2：后面（-Z 方向）- 数字 1

- 顶点顺序：$V_1 \to V_0 \to V_3 \to V_2$（从后面看逆时针）
- 数字编号：1

- 纹理范围：$u \in [0.125, 0.25]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_1	$(+1, -1, -1)$	$(0.125, 0)$	左下
V_0	$(-1, -1, -1)$	$(0.250, 0)$	右下
V_3	$(-1, +1, -1)$	$(0.250, 1)$	右上
V_2	$(+1, +1, -1)$	$(0.125, 1)$	左上

面 3：左面（-X 方向）- 数字 2

- 顶点顺序：$V_0 \to V_4 \to V_7 \to V_3$（从左面看逆时针）
- 数字编号：2
- 纹理范围：$u \in [0.25, 0.375]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_0	$(-1, -1, -1)$	$(0.250, 0)$	左下
V_4	$(-1, -1, +1)$	$(0.375, 0)$	右下
V_7	$(-1, +1, +1)$	$(0.375, 1)$	右上
V_3	$(-1, +1, -1)$	$(0.250, 1)$	左上

面 4：右面（+X 方向）- 数字 3

- 顶点顺序：$V_5 \to V_1 \to V_2 \to V_6$（从右面看逆时针）
- 数字编号：3
- 纹理范围：$u \in [0.375, 0.5]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_5	$(+1, -1, +1)$	$(0.375, 0)$	左下
V_1	$(+1, -1, -1)$	$(0.500, 0)$	右下
V_2	$(+1, +1, -1)$	$(0.500, 1)$	右上
V_6	$(+1, +1, +1)$	$(0.375, 1)$	左上

面 5：上面（+Y 方向）- 数字 4

- 顶点顺序：$V_3 \to V_7 \to V_6 \to V_2$（从上面看逆时针）
- 数字编号：4
- 纹理范围：$u \in [0.5, 0.625]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_3	$(-1, +1, -1)$	$(0.500, 0)$	左下
V_7	$(-1, +1, +1)$	$(0.625, 0)$	右下
V_6	$(+1, +1, +1)$	$(0.625, 1)$	右上
V_2	$(+1, +1, -1)$	$(0.500, 1)$	左上

面 6：下面（-Y 方向）- 数字 5

- 顶点顺序：$V_4 \to V_0 \to V_1 \to V_5$（从下面看逆时针）
- 数字编号：5
- 纹理范围：$u \in [0.625, 0.75]$, $v \in [0, 1]$

顶点	位置坐标	纹理坐标	位置
V_4	$(-1, -1, +1)$	$(0.625, 0)$	左下
V_0	$(-1, -1, -1)$	$(0.750, 0)$	右下
V_1	$(+1, -1, -1)$	$(0.750, 1)$	右上
V_5	$(+1, -1, +1)$	$(0.625, 1)$	左上

3.4 纹理坐标汇总表

表 3: 正方体六个面纹理坐标汇总

面	方向	数字	左下	右下	右上	左上
前面	+Z	0	(0.000, 0)	(0.125, 0)	(0.125, 1)	(0.000, 1)
后面	-Z	1	(0.125, 0)	(0.250, 0)	(0.250, 1)	(0.125, 1)
左面	-X	2	(0.250, 0)	(0.375, 0)	(0.375, 1)	(0.250, 1)
右面	+X	3	(0.375, 0)	(0.500, 0)	(0.500, 1)	(0.375, 1)
上面	+Y	4	(0.500, 0)	(0.625, 0)	(0.625, 1)	(0.500, 1)
下面	-Y	5	(0.625, 0)	(0.750, 0)	(0.750, 1)	(0.625, 1)

3.5 代码实现示例

```rust
/// 二维向量，用于纹理坐标
#[derive(Debug, Clone, Copy)]
pub struct Vec2 {
    pub u: f32,
    pub v: f32,
}

/// 三维向量，用于位置坐标
#[derive(Debug, Clone, Copy)]
pub struct Vec3 {
    pub x: f32,
    pub y: f32,
    pub z: f32,
}

/// 正方体顶点（含纹理坐标）
#[derive(Debug, Clone, Copy)]
pub struct CubeVertex {
    pub position: Vec3,
    pub tex_coord: Vec2,
}

/// 生成正方体顶点数据（每个面4个顶点，共24个）
/// 因为同一顶点在不同面有不同纹理坐标
pub fn create_cube_vertices() -> [CubeVertex; 24] {
    [
        // 前面 (+Z) - 数字0
        CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: 1.0 },
                     tex_coord: Vec2 { u: 0.000, v: 0.0 } },  //
                        V4 左下
        CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: 1.0 },
                     tex_coord: Vec2 { u: 0.125, v: 0.0 } },  //
                        V5 右下
        CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: 1.0 },
                     tex_coord: Vec2 { u: 0.125, v: 1.0 } },  //
                        V6 右上
        CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: 1.0 },
                     tex_coord: Vec2 { u: 0.000, v: 1.0 } },  //
                        V7 左上
```

```
36
37          // 后面 (-Z) - 数字1
38          CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: -1.0 },
39                       tex_coord: Vec2 { u: 0.125, v: 0.0 } },  //
                            V1 左下
40          CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: -1.0
               },
41                       tex_coord: Vec2 { u: 0.250, v: 0.0 } },  //
                            V0 右下
42          CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: -1.0 },
43                       tex_coord: Vec2 { u: 0.250, v: 1.0 } },  //
                            V3 右上
44          CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: -1.0 },
45                       tex_coord: Vec2 { u: 0.125, v: 1.0 } },  //
                            V2 左上
46
47          // 左面 (-X) - 数字2
48          CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: -1.0
               },
49                       tex_coord: Vec2 { u: 0.250, v: 0.0 } },  //
                            V0 左下
50          CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: 1.0 },
51                       tex_coord: Vec2 { u: 0.375, v: 0.0 } },  //
                            V4 右下
52          CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: 1.0 },
53                       tex_coord: Vec2 { u: 0.375, v: 1.0 } },  //
                            V7 右上
54          CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: -1.0 },
55                       tex_coord: Vec2 { u: 0.250, v: 1.0 } },  //
                            V3 左上
56
57          // 右面 (+X) - 数字3
58          CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: 1.0 },
59                       tex_coord: Vec2 { u: 0.375, v: 0.0 } },  //
                            V5 左下
60          CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: -1.0 },
61                       tex_coord: Vec2 { u: 0.500, v: 0.0 } },  //
                            V1 右下
62          CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: -1.0 },
63                       tex_coord: Vec2 { u: 0.500, v: 1.0 } },  //
```

```
                        V2 右上
64          CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: 1.0 },
65                       tex_coord: Vec2 { u: 0.375, v: 1.0 } },  //
                            V6 左上
66
67          // 上面 (+Y) - 数字4
68          CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: -1.0 },
69                       tex_coord: Vec2 { u: 0.500, v: 0.0 } },  //
                            V3 左下
70          CubeVertex { position: Vec3 { x: -1.0, y: 1.0, z: 1.0 },
71                       tex_coord: Vec2 { u: 0.625, v: 0.0 } },  //
                            V7 右下
72          CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: 1.0 },
73                       tex_coord: Vec2 { u: 0.625, v: 1.0 } },  //
                            V6 右上
74          CubeVertex { position: Vec3 { x: 1.0, y: 1.0, z: -1.0 },
75                       tex_coord: Vec2 { u: 0.500, v: 1.0 } },  //
                            V2 左上
76
77          // 下面 (-Y) - 数字5
78          CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: 1.0 },
79                       tex_coord: Vec2 { u: 0.625, v: 0.0 } },  //
                            V4 左下
80          CubeVertex { position: Vec3 { x: -1.0, y: -1.0, z: -1.0
               },
81                       tex_coord: Vec2 { u: 0.750, v: 0.0 } },  //
                            V0 右下
82          CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: -1.0 },
83                       tex_coord: Vec2 { u: 0.750, v: 1.0 } },  //
                            V1 右上
84          CubeVertex { position: Vec3 { x: 1.0, y: -1.0, z: 1.0 },
85                       tex_coord: Vec2 { u: 0.625, v: 1.0 } },  //
                            V5 左上
86      ]
87  }
88
89  /// 生成三角形索引（每个面2个三角形，共36个索引）
90  pub fn create_cube_indices() -> [u32; 36] {
91      [
92          0, 1, 2,  0, 2, 3,       // 前面
```

```
93            4, 5, 6,  4, 6, 7,       // 后面
94            8, 9, 10, 8, 10, 11,     // 左面
95            12, 13, 14, 12, 14, 15, // 右面
96            16, 17, 18, 16, 18, 19, // 上面
97            20, 21, 22, 20, 22, 23, // 下面
98        ]
99    }
```

Listing 8: 正方体纹理坐标定义

3.6 注意事项

1. **顶点重复：**由于同一个几何顶点在不同面上需要不同的纹理坐标，因此正方体需要 24 个顶点（而非 8 个）来存储完整的纹理映射信息。

2. **纹理方向：**需要确保纹理在每个面上的方向正确，避免数字显示颠倒或镜像。

3. **纹理过滤：**在纹理边界处可能出现采样问题，建议在数字之间留出少量边距，或使用 GL_CLAMP_TO_EDGE 等纹理包裹模式。

4. **面的朝向：**顶点顺序决定了面的法向量方向，需要保持一致的绑定顺序（如逆时针为正面）。